1. Introduction

This project is a simple Library Book Management System developed in Python.

It maintains a list of books and allows the user to perform operations like adding, viewing, issuing, returning, searching, and deleting books.

2. Features

- Add a new book
- View all books
- Issue a book
- Return a book
- Search book by title
- Remove/Delete book
- Exit the program

3. Data Structures Used

books = []

Stores all book details in a list.

Each book is a dictionary containing ID, title, author, and status.

book_id_counter = 1

Helps assign unique IDs to each book.

4. Function Explanations (Line-by-Line)

4.1 add_book()

def add_book():

global book_id_counter

```
title = input("Enter book title: ")
author = input("Enter book author: ")

book = {
"id": book_id_counter,
"title": title,
"author": author,
"status": "available"
}
books.append(book)
book_id_counter += 1
print("Book added successfully!\n")
```

Explanation:

- Uses global counter
- Takes title and author
- Creates dictionary
- Adds book
- Increases ID

4.2 view_books()

```
def view_books():
if not books:
print("No books available.\n")
return

print("\n--- Library Books ---")
for book in books:
```

print(f"ID: {book['id']} | Title: {book['title']} | Author: {book['author']} | Status: {book['status']}")

print()

Explanation:

• Checks if empty

• Prints all books

4.3 issue_book()

def issue_book():

book_id = int(input("Enter book ID to issue: "))

for book in books:

if book['id'] == book_id:

if book['status'] == "issued":

print("Book is already issued!\n")

else:

book['status'] = "issued"

print("Book issued successfully!\n")

return

print("Book ID not found!\n")

Explanation:

• Takes ID

• Searches

• If issued → warning

• Else → issue

• If not found → error

4.4 return_book()

```
def return_book():

book_id = int(input("Enter book ID to return: "))

for book in books:

if book['id'] == book_id:

if book['status'] == "available":

print("Book is already available in library!\n")

else:

book['status'] = "available"

print("Book returned successfully!\n")

return

print("Book ID not found!\n")
```

Explanation:

• Takes ID

• If already available → warning

• Else → mark available

• If not found → error

4.5 search_book()

```
def search_book():

search_title = input("Enter book title to search: ").lower()

found = False

for book in books:

if search_title in book['title'].lower():

print(f"ID: {book['id']} | Title: {book['title']} | Author: {book['author']} | Status: {book['status']}")
```

found = True

if not found:

print("No matching book found!\n")

Explanation:

• Input title

• Converts to lowercase

• Searches partial match

• If none found → message

4.6 remove_book()

def remove_book():

book_id = int(input("Enter book ID to delete: "))

for book in books:

if book['id'] == book_id:

books.remove(book)

print("Book deleted successfully!\n")

return

print("Book ID not found!\n")

Explanation:

• Takes ID

• Finds book

• Removes

• If not found → error

5. Main Program Loop

```
while True:

print("""

======== LIBRARY MANAGEMENT ========

1. Add Book

2. View All Books

3. Issue Book

4. Return Book

5. Search Book by Title

6. Remove/Delete Book

7. Exit Program

===================================

""")

choice = input("Enter your choice: ")

if choice == "1":

add_book()

elif choice == "2":

view_books()

elif choice == "3":

issue_book()

elif choice == "4":

return_book()

elif choice == "5":

search_book()

elif choice == "6":
```

```
remove_book()

elif choice == "7":

print("Exiting program... Goodbye!")

break

else:

print("invalid choice,try again!\n")
```

Explanation:

- Displays menu
- Based on input, calls functions
- 7 exits program

6. Conclusion

This project demonstrates lists, dictionaries, loops, functions, and conditional statements to build a basic Python library management system.